SmartLock v1: Functionality and Protocol Reference

SmartLock is a small Internet-of-Things product: a smart door lock (the **device**, which runs the server) controlled by a home **hub** (the controller, which runs the client). This document is the vendor-style reference you are given as the starting point. It describes, in plain terms, what the product does and how its network protocol works. It deliberately says nothing about whether the design is secure. Judging that, stage by stage, is your job.

1. What the product does

A home owner installs the lock on a door and keeps the hub somewhere central in the house. The hub talks to the lock over the local network. In normal use the hub:

1. **Connects** to the lock over TCP.
2. **Agrees a cipher suite** with it (which encryption to use for the session).
3. **Checks the lock's identity** by reading the lock's certificate.
4. **Pairs** with the lock by proving it knows the pairing PIN printed on the device label.
5. **Agrees a shared key** with the lock using a Diffie-Hellman exchange.
6. **Sends commands** such as "what is your status", "unlock", and "lock", each one encrypted as a record.
7. **Closes** the session.

The lock keeps two pieces of state that make the effect of every command visible:

- a **door state**, either `LOCKED` or `UNLOCKED`, and
- an **unlock counter** that increases by one every time the door is opened.

Both last for as long as `lock_server` runs. They carry over from one session to the next and reset only when you restart it.

When the session key has been agreed, the lock sends a one-line **greeting** that contains the device's **pairing secret**. In this assignment that pairing secret is your **flag**: a string of the form `FLAG{...}` that is unique to your instance and that you will try to recover as an attacker.

2. The parts you are given

`slink.hpp`	Shared protocol helpers: the wire framing, the cipher-suite table, the (small) Diffie-Hellman arithmetic, and the record encryption and decryption. Header only.
`pki.hpp`	RSA helpers built on OpenSSL: load a key, sign a message, verify a signature.
`lock_server.cpp`	The lock device. It loads your instance file and serves one hub session at a time.

`hub_client.cpp`	A legitimate hub. It runs a complete, well-behaved session so you can see normal traffic.
`instance_generator.cpp`	Produces your `instance.txt` from your roll number. You run it once.
`secrets.hpp`	The deterministic derivation the generator uses (seeded from your roll).

`pin_wordlist.txt`

	A dictionary for the pairing PIN (a word followed by three digits), for use with hashcat or John.
`Makefile`	Builds `lock_server` , `hub_client` , and `instance_generator` .

You create `instance.txt` yourself (see the next section); it is your instance: the lock's configuration and your pairing secret.

3. Building and running it

The code is C++17 and links OpenSSL's `libcrypto` . On Kali, Debian, or Ubuntu, install the toolchain once:

```
sudo apt install build-essential libssl-dev
```

Build the programs, then generate your instance once from your roll number:

```
cd smartlock
make
./instance_generator 23i-0001 # use your own roll; writes instance.txt
```

You now have `lock_server` , `hub_client` , and your `instance.txt` . Start the lock in one terminal and the hub in another. On a single machine for a quick smoke test, use `127.0.0.1` ; in the two VM lab (see the Lab Setup Guide) use the lock VM's address.

```
# terminal 1 (the lock / device)
./lock_server instance.txt

# terminal 2 (the hub / controller)
./hub_client instance.txt 127.0.0.1
```

The hub connects to the port in `instance.txt` . To send it somewhere else, such as a relay in the lab, add a port: `./hub_client instance.txt <host> <port>` .

A worked example of a normal session

With a sample instance, the lock prints its listening port and the hub prints the conversation. Your numbers, name, and flag differ because your instance is unique.

```
$ ./lock_server instance.txt
SmartLock smartlock-23i-0001.local listening on port 9257
```

```
$ ./hub_client instance.txt 127.0.0.1
[hub] negotiated suite: AES256-GCM
[hub] device claims to be: smartlock-23i-0001.local (certificate not checked)
[lock] Paired with SmartLock. Pairing secret: FLAG{xxxxxxxxxxxxxxx}
[lock] STATE LOCKED unlocks=0
[lock] DOOR OPENED (unlock #1)
[lock] DOOR LOCKED
```

This is the first session after starting the lock. Run the hub again and its `STATUS` reply shows `unlocks=1`, then `unlock #2`, because the counter carries over.

Read the greeting line carefully: the lock hands the pairing secret to the hub as soon as the session key is agreed. In a correct system that line is protected so that only the paired hub can read it. Part of your task is to show whether that holds here.

4. How the protocol works (the wire format)

The handshake and the records run over a single, plain TCP connection. **Every message is one line**, ending in a newline. Within a line, fields are separated by a TAB character, and each field is written as `key:value`. The values are plain decimal, hexadecimal, or base64 text (none of them contains a TAB or a colon), so if you capture the connection you can read the entire exchange as text. The messages appear in this fixed order.

Stage 1, negotiation.

- The hub sends `type:HELLO` with `suites.<comma-separated list>`, strongest first. The supported suite names are `AES256-GCM`, `AES128-GCM`, and `NULL` (the last meaning no record encryption at all).
- The lock replies `type:SUITE` with `suite.<the one it chose>`.

Stage 2, authentication.

- The lock sends `type:CERT` with `name` (the device name), `pubkey` (its RSA public key as base64-encoded PEM), and `sig` (a signature, in hex).
- The hub sends `type:PAIR` with `hub` (an identifier) and `pinhash`, where `pinhash` is a digest of the pairing PIN.

Stage 3, keying.

- The lock sends `type:DH` with `p`, `g`, and `B` (the Diffie-Hellman prime, generator, and the lock's public value).
- The hub sends `type:DHCLIENT` with `A` (the hub's public value).
- Each side then combines its own secret with the other side's public value to reach the same shared secret, and turns that into the record key. The chosen suite fixes the key length: 32 bytes for `AES256-GCM`, 16 for `AES128-GCM`.

Stage 4, message protection.

- Each record is `type:MSG` with `nonce` (hex) and `ct` (hex). Under a GCM suite the ciphertext carries a 16-byte authentication tag at its end. Under `NULL` the field is simply the plaintext.
- The commands carried inside records are `STATUS`, `UNLOCK`, `LOCK`, and `QUIT`.

Stage 5, teardown.

- The hub sends `QUIT` inside a record, and the session then ends.

The whole exchange at a glance

This is one session, in order. Every line is plain text on the wire until the records begin; the records are ciphertext, but the `nonce` beside each one is sent in clear.

1	hub to lock	`HELLO`	`suites`	the full list of offered suites, including `NULL`, in clear
2	lock to hub	`SUITE`	`suite`	the chosen suite, in clear
3	lock to hub	`CERT`	`name`, `pubkey`, `sig`	the device name, its RSA public key, and a signature, in clear

4	hub to lock	`PAIR`	`hub`, `pinhash`	the hub id and the MD5 digest of the PIN, in clear
5	lock to hub	`DH`	`p`, `g`, `B`	the group and the lock's public value, in clear
6	hub to lock	`DHCLIENT`	`A`	the hub's public value, in clear
7	lock to hub	`MSG` (greeting)	`nonce`, `ct`	the static nonce in clear; the encrypted greeting that carries the flag
8	hub to lock	`MSG`	`nonce`, `ct`	the same nonce again; an encrypted command (`STATUS`, `UNLOCK`, `LOCK`, `QUIT`)
9	lock to hub	`MSG`	`nonce`, `ct`	the same nonce again; the encrypted reply (the state and the counter)

Steps 8 and 9 repeat for each command. Note that the `nonce` field in steps 7 to 9 never changes within a session.

How the record key is derived

Both sides reach the same shared secret from the Diffie-Hellman exchange: the hub computes `s = B^a mod p` and the lock computes `s = A^b mod p`, and these are equal. v1 then turns `s` **straight into the record key** with no key derivation function: it writes `s` as big-endian bytes, right-aligned in a zero-filled buffer whose length is set by the suite (32 bytes for `AES256-GCM`, 16 for `AES128-GCM`). This is `session_key()` in `slink.hpp`. The nonce is

the fixed `static_nonce` from the instance, reused for every record. So anyone who learns `s` (for example by solving the discrete log for `b` and combining it with the captured `A`) can rebuild the exact key and nonce the two parties used, and read or forge records.

Behaviour you can rely on while testing

- The greeting is sent **after** the Diffie-Hellman exchange finishes, not immediately after pairing, because it travels as an encrypted record.
- The hub is **not interactive**: it runs a fixed script (read the greeting, then `STATUS`, `UNLOCK`, `LOCK`, `QUIT`) and exits. To send other commands, write your own client.
- The lock serves **one session at a time**. A second hub waits until the first connection closes.
- The door state and the unlock counter **persist for the life of the lock process** and reset only when you restart `lock_server`.

5. The five stages as a checklist

The whole engagement is organised around the five stages above. For each one, ask the questions a reviewer would ask:

- **Negotiation:** can an outsider influence which suite is chosen? Is the choice protected once made?
- **Authentication:** does the hub actually verify who it is talking to? How is the PIN protected on the wire, and how hard is it to recover?
- **Keying:** how strong is the key agreement, and is the key derived properly from the shared secret? Are past sessions safe if a key later leaks?
- **Message protection:** is each message confidential, authentic, fresh, and in order?

- **Teardown:** is the end of a session protected? Can an old message be replayed?

Keep this reference beside you while you read the source. The Appendix explains the attacks you are likely to try, and the assignment statement sets out exactly what to submit.